

Lab 6 – Secure Testing using SAST Tool

You have developed a web-based student portal for a university (In Lab 5). The portal allows students to:

- Log in to view grades and attendance
- Submit assignments
- Communicate with lecturers via messaging
- Update personal information

Continue: Scenario

Excellent work. Your student portal application is now successfully developed and running. The next step is to test the application using a SAST tool. In this activity, you will scan the source code to look for possible security weaknesses, review the findings, and understand how SAST helps developers detect security issues early before deployment.

Step 1 — Confirm the secured application is working properly

Before scanning, verify:

- login works with valid credentials
- wrong login returns generic error
- assignment upload accepts allowed file types
- messaging works
- profile update only edits the logged-in user profile

This is important because SAST scans code, but DAST later needs a working live app.

Step 2 — Install Semgrep for SAST

Make sure your venv active (if you use env when setup):

- `pip install semgrep`

Run a scan in the project folder:

`semgrep scan .`

```
(venv)-(niksa@niksa)-[~/student_portal_lab_secure]
$ semgrep scan . --json

Semgrep CLI

Loading rules from registry...
Scanning 5753 files (only git-tracked) with:
✓ Semgrep OSS
  ✓ Basic security coverage for first-party code
  ✗ Do not use it in a production environment. Instead, use Semgrep Code (SAST) for advanced scanning and expert security rules.
✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with Semgrep Code (SAST) for advanced scanning and expert security rules.
✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.
💡 Get started with all Semgrep products via 'semgrep login'.
🌟 Learn more at https://sg.run/cloud.

100% 0:05:13

{"version":"1.157.0","results":[{"check_id":"python.flask.security.audit.render-template-string.render-template-string","path":"app_secure.py","start":{"l
```

OR

scan only the app file:

`semgrep scan app_secure.py`

```
(venv)-(niksa@niksa)-[~/student_portal_lab_secure]
$ semgrep scan app_secure.py

Semgrep CLI

Loading rules from registry...
Scanning 1 file (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code
  vulnerabilities.

✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with secure
  advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS
  dependencies.

Get started with all Semgrep products via 'semgrep login'.
Learn more at https://sg.run/cloud.

100% 0:00:00
6 Code Findings

app_secure.py
python.flask.security.audit.render-template-string.render-template-string
```

This checks the source code for insecure patterns.

Example: `semgrep scan --config auto . --json -o semgrep-results.json`

It build the summary from:

- total findings
- findings by severity
- affected files
- most important rule IDs
- short remediation focus

Your Task:

- 1- Create a proper detail report from this *semgrep* tool. (Example; View and export the report details nicely in a file using JSON, etc.)
- 2- Fill up the form attached with report detail and submit to Moodle.